

HARVARD UNIVERSITY EXTENSION SCHOOL



Capstone Proposal

Agentic Gyms & Prompt Optimization

Submitted by:

Michael Culver

Jinyu Han

Jerry Kim

Matthew Michel

Kaylee Vo

January 27, 2026

Contents

1	Introduction	3
1.1	Context	3
1.1.1	The Optimization Landscape	3
1.1.2	Gym Environment Selection	4
1.1.3	Research Objective	4
1.2	Problem & Setting	4
1.2.1	Research Gap	5
1.3	Subproblems	5
1.3.1	Primary Research Question	5
1.3.2	Specific Subproblems	5
1.4	Theoretical & Conceptual Framework	5
1.4.1	Sequential Decision Theory	5
1.4.2	Evolutionary Computation	6
1.4.3	Credit Assignment	6
1.4.4	Trajectory-Aware Fitness	6
1.5	A Priori Hypotheses	7
1.6	Variables and Key Concepts	8
1.7	Assumptions, Delimitations, and Limitations	8
1.7.1	Assumptions	8
1.7.2	Delimitations	8
1.7.3	Limitations	9
1.8	Significance of the Study	9
1.8.1	Democratization of Agent Training	9
1.8.2	Technical Contribution	10
2	Literature Review	11
2.1	Reinforcement Learning Methods and Gym Environments	11
2.1.1	Foundations of RL for LLM Agents	12
2.1.2	Preference-Based and Outcome-Driven Methods	12
2.1.3	Credit Assignment in Multi-Turn Settings	13
2.1.4	Gym Environments for Agentic Evaluation	13
2.2	Prompt Engineering and Evolutionary Methods	14
2.2.1	Structured Reasoning as a Token-Space Foundation	14
2.2.2	Automated Prompt Optimization	15
2.2.3	Evolutionary Prompt Optimization	16

2.2.4	Limitations for Multi-Turn Agentic Tasks	16
2.3	Bridging Gyms and Prompt Optimization	17
2.3.1	Credit Assignment in Token Space	18
2.3.2	Multi-Objective Optimization	18
2.3.3	Dense Feedback Mechanisms	19
2.3.4	Structured Agent Patterns	19
2.4	Literature Review Conclusion	20
3	Methodology	21
3.1	Research Design	21
3.1.1	Comparative Framework	21
3.1.2	Gym Environment Selection	21
3.1.3	Experimental Conditions	22
3.1.4	Task Environments and Evaluation Protocols	22
3.2	Technical Components	22
3.2.1	GEM-DSPy Adapter	23
3.2.2	GEPA Optimizer Configuration	23
3.2.3	AWS Infrastructure	24
3.3	Metrics and Model Evaluations	26
3.3.1	Primary Metrics	26
3.3.2	Secondary Metrics	27
3.3.3	Baseline Specifications	27
3.4	Data Collection	28
3.4.1	Data Sources	28
3.4.2	Data Capture Methods	28
3.5	Sample and Sampling Procedures	29
3.5.1	Task Sampling	29
3.5.2	Episode Sampling	29
3.5.3	Randomization	29
3.6	Ethical Considerations	30
3.7	Validity and Reliability	30
3.8	Methodological Limitations	30
3.9	Additional Objectives (Stretch Goals)	30
3.10	Project Timeline	31
3.10.1	Risk Mitigation	32
	References	33

Chapter 1

Introduction

1.1 Context

Since 2022, large language models (LLMs) have shifted from isolated question-answering to complex multi-step workflows. Agentic architectures—incorporating memory persistence, tool use, and iterative planning—serve as scaffolding that enables this transition. However, these systems remain inconsistent and unpredictable. For enterprises, this unpredictability translates to hesitancy in deploying agentic systems for mission-critical workflows, limiting potential productivity gains and return on AI investments.

The core barrier is cost: dominant research paradigms prioritize scaling laws and post-training refinement of base models, which is resource-intensive. Organizations need systematic, cost-effective methods to understand and improve agent performance. This research investigates whether prompt optimization methods, operating in token space rather than weight space, can achieve comparable results to reinforcement learning.

1.1.1 The Optimization Landscape

To address this challenge, organizations are exploring systematic methods for improving agentic performance. We conceptualize this landscape as operating in three spaces: weight space (RL modifying model parameters), activation space (modifying internal representations (Sinii et al., 2025)), and token space (prompt optimization modifying input context). Weight-space methods like PPO and GRPO operate at **training time**, requiring significant upfront compute but no inference overhead. Token-space methods operate at **inference time**, prepending optimized instructions to each query—offering flexibility but accumulating per-call cost.

In the weight-space paradigm, “gyms” modeled after OpenAI Gym (Brockman et al., 2016) provide standardized environments for RL training. Token-space optimizers like DSPy (Khatab et al., 2024) and GEPA (Agrawal et al., 2025) instead operate directly on task datasets, using evaluation metrics as optimization signals. This simplicity is advantageous, but raises the question of whether such methods can handle the multi-turn reasoning that gym environments are designed to test.

1.1.2 Gym Environment Selection

To conduct this comparison, we require a standardized environment that supports both paradigms. We selected GEM (General Experience Maker) (Z. Liu et al., 2025), an open-source gym for agentic LLMs that provides per-turn rewards and published RL baselines. The rationale for this selection is detailed in Section 3.1.1.

1.1.3 Research Objective

This research aims to determine whether lightweight prompt optimization can achieve comparable performance to reinforcement learning on agentic tasks—while requiring substantially less computational investment. If successful, this would lower the barrier to entry for organizations seeking to deploy effective agentic systems, shifting agent optimization from a resource-intensive endeavor to an accessible engineering practice.

1.2 Problem & Setting

Gym-based training environments have been utilized primarily with computationally heavy reinforcement learning methods, while prompt optimization techniques have been limited to single-turn tasks. No direct empirical comparison exists between these paradigms on identical multi-turn agentic tasks—leaving practitioners without the evidence needed to make informed optimization decisions.

While promising frameworks exist on both sides—GEM providing standardized gym environments with per-turn rewards, and prompt optimizers like GEPA offering reflective evolution capabilities—no integration between these paradigms currently exists. Without direct benchmarks on identical tasks, organizations lack the evidence needed to justify either the upfront investment in RL infrastructure or the operational costs of inference-time prompt optimization.

	Single-Step Tasks	Complex Workflows
High Cost	Inefficient Zone: Early-stage RL experiments on simple tasks.	Deep RL-based Alignment: PPO (Proximal Policy Optimization), GRPO (Group Relative Policy Optimization), REINFORCE
Low Cost	Prompt Optimization: DSPy (Traditional usage), Basic Prompt Engineering	Trajectory-Aware Optimization: GEPA (Genetic-Pareto), GEM-DSPy (Proposed Integration)

Figure 1.1: Cost vs. task complexity landscape for optimization approaches

1.2.1 Research Gap

This study addresses the gap by providing: (1) a functional integration of gym environments with prompt optimization frameworks and (2) an empirical comparison of prompt-optimized agents versus RL-optimized agents for question-answer tasks, MCP and docker tasks.

1.3 Subproblems

1.3.1 Primary Research Question

Can prompt optimization, specifically GEPA (Genetic-Pareto), achieve an agentic task success rate comparable to a reinforcement learning baseline while requiring fewer resources? The model performance will be determined by repeated experiments on the tasks mentioned in 2.1. This will provide us with a confidence interval to establish statistical significance on improvements, if any. Any large model assumptions will have a corresponding sensitivity to ensure results hold. Compute performance will be measured using wall-clock time and estimated dollar cost reported as secondary measures.

1.3.2 Specific Subproblems

1. **Evaluation Benchmarks:** What metrics and statistical methods are appropriate for comparing optimization approaches with fundamentally different cost structures (upfront training vs. per-inference overhead)?
2. **Fitness Function Design:** Can per-turn rewards be effectively leveraged as a prompt optimization fitness signal to enable fine-grained credit assignment comparable to RL reward shaping?
3. **Composite Metrics:** Can fitness functions be extended to include behavioral dimensions (e.g., loop detection, step efficiency) that capture agent quality beyond task success rate?
4. **Generalization:** Do optimized prompts transfer across task variations within an environment, or do they overfit to the specific examples seen during optimization?

1.4 Theoretical & Conceptual Framework

This study draws on three theoretical areas: **Sequential Decision Theory**, **Evolutionary Computation**, and **Credit Assignment**.

1.4.1 Sequential Decision Theory

Both paradigms operate on agents formalized as decision-making processes. RL models agents as Markov Decision Processes (MDPs) where the agent observes state s_t , takes action a_t , receives reward r_t , and transitions to s_{t+1} (Sutton & Barto, 2018). While text-based

environments are technically Partially Observable MDPs (POMDPs) (Kaelbling et al., 1998), we adopt the standard MDP formalization as a simplifying assumption, positing that chain-of-thought reasoning (Wei et al., 2022) serves as implicit working memory that mitigates partial observability.

1.4.2 Evolutionary Computation

GEPA treats candidate prompts as individuals undergoing selection, mutation, and recombination (Agrawal et al., 2025), navigating a *fitness landscape* mapping prompts to performance scores. The No Free Lunch theorem motivates our empirical comparison: no algorithm dominates universally, so prompt optimization may outperform RL on certain problem structures.

1.4.3 Credit Assignment

A central challenge is *credit assignment*, determining which decisions contributed to final outcomes (Ferret et al., 2023). RL addresses this through temporal difference learning; prompt optimization traditionally lacks those specific temporal differencing mechanisms. Our *trajectory-aware fitness* leverages GEM’s per-turn rewards to score intermediate steps, naturally importing credit assignment into the prompt optimization paradigm.

1.4.4 Trajectory-Aware Fitness

Unlike single-turn optimization, trajectory-aware fitness evaluates prompts on complete episode traces, which follow a Markov Decision Process. A trajectory consists of state-action-reward transitions:

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T)$$

where:

- s_t : state at time t
- a_t : action taken from state s_t
- r_{t+1} : reward received after taking action a_t
- T : episode horizon

A prompt p induces a policy π_p over the LLM’s action space, which is equivalent to the probability distribution of a prompt parameterized by Ω . The fitness of prompt p is given by $F(\tau)$. The goal is to find the best set of prompt tokens Ω by maximizing the expected fitness $F(\tau)$ across all trajectories.

$$\max_{\Omega} \mathbb{E}_{\tau \sim p_{\Omega}(\tau)} [F(\tau)], \quad \text{where } F(\tau) = \sum_{t=0}^{T-1} g(\gamma, t, r_t) \quad (1.1)$$

The term $\gamma \in [0, 1]$ is a discount factor, where 0 values only the immediate reward (near-sighted) and 1 values the future rewards equally to present rewards (far-sighted). If $F(\tau_1) >$

$F(\tau_2)$, then τ_1 is the better trajectory. g is arbitrary function of γ, t and r_t . It can be defined to weigh rewards evenly, give more importance to states near the end of a trajectory, adjust the weight on the final state reward, and so forth.

This formulation establishes a common optimization objective for both paradigms: RL fine-tuning optimizes the discounted return via gradient updates to continuous weight parameters, while evolutionary prompt search optimizes the same objective via population-based search over prompts.

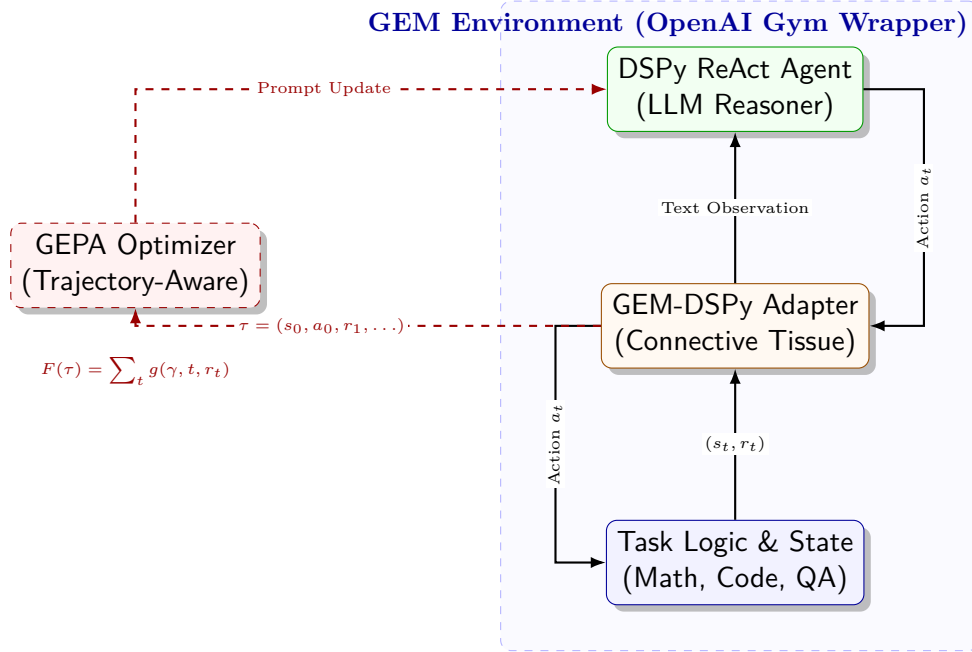


Figure 1.2: Conceptual Framework: The GEM-DSPy Adapter serves as the connective tissue between the internal Gym execution loop and the external GEPA optimizer. The agent operates within the Gym boundary, receiving evolutionary prompt updates based on captured trajectories.

1.5 A Priori Hypotheses

- **H1 (Performance):** On each GEM environment (Math12K, CodeContest, HotpotQA), the GEPA-optimized agent will achieve a task success rate within 5 percentage points of the RL baseline under matched evaluation budgets and decoding settings. Equivalence will be assessed using Two One-Sided Tests (TOST) at $\alpha = 0.05$. Sensitivity analysis will be conducted on model assumptions to ensure any measurable difference (Δ) is robust.
- **H2 (Compute/Cost):** GEPA optimization will require at least one order of magnitude fewer optimization resources (measured in wall-clock time and estimated dollar cost) than the RL baseline’s training procedure, while achieving comparable task performance as defined in H1.

- **H3 (Behavioral Mechanism):** The incorporation of composite fitness terms (loop detection and step efficiency) significantly improves GEPA convergence rate and final performance, serving as a mechanism to prune unproductive trajectories in token space. This will be tested via ablation: removing these terms is expected to degrade convergence speed or task success rate by a measurable margin.

1.6 Variables and Key Concepts

Independent Variable: Optimization method (RL via PPO/GRPO vs. prompt optimization via GEPA). Task environments, base models, and evaluation protocols are held constant to isolate the effect of the optimization approach.

Dependent Variables:

- Task Success Rate (Accuracy).
- Compute Cost (Cost of API calls and training duration).
- Trajectory Efficiency (Number of steps to solution).

Key Concepts:

- **GEM (General Experience Maker):** An environment providing dense (per-turn) reward signals and $\gamma < 1$ discount factors (Z. Liu et al., 2025).
- **ReAct:** A framework that synergizes reasoning and acting in LLMs by interleaving reasoning traces with task-specific actions (Yao, Zhao, et al., 2023).
- **GEPA (Genetic-Pareto):** A reflective prompt evolution algorithm that learns rules via trial and error (Agrawal et al., 2025).

1.7 Assumptions, Delimitations, and Limitations

1.7.1 Assumptions

We assume that GEM’s string-based observation space is sufficient for DSPy modules to infer state. While standard gym environments often provide complete state information, text-based environments are effectively Partially Observable Markov Decision Processes (POMDPs). We posit that the LLM’s reasoning traces (Chain of Thought (Wei et al., 2022)) serve as an implicit working memory that mitigates this partial observability, allowing effective operation without explicit memory vectors.

1.7.2 Delimitations

This study is delimited to three specific environments within GEM: Math12K, CodeContest, and HotpotQA. It will not explore visual agents or multi-modal inputs, nor will it assess various types of ‘agentic scaffolding’ e.g. memory or guardrails optimization.

1.7.3 Limitations

The study relies on API-based LLMs (e.g., OpenAI or Anthropic). Variations in API behavior, pricing, and model updates during the 16-week timeline may introduce noise in latency and cost measurements. Because GEPA is an optimization method, there is a risk of overfitting; we mitigate this with strict held-out test sets not accessed during optimization.

This study compares GEPA and RL under specific conditions: fixed compute budgets, three task environments, and particular hyperparameter configurations. If GEPA fails to match RL performance, this would not definitively establish prompt optimization as inferior—alternative explanations include suboptimal tuning or task-specific characteristics favoring weight-space methods. Hybrid approaches (e.g., GEPA-optimized prompts as RL initialization) fall outside the current scope.

For fair comparison, we use Qwen3 models (matching GEM’s RL experiments) via AWS infrastructure for both methods. We also report results using Claude Sonnet 4.5 as a secondary comparison to assess generalization across model families.

1.8 Significance of the Study

1.8.1 Democratization of Agent Training

Current agent training is cost-prohibitive; reinforcement learning methods require substantial GPU infrastructure, specialized engineering expertise, and large reward-modeling pipelines. Typically, only organizations with significant R&D budgets can afford it. If GEPA can match RL performance, it validates a workflow requiring no GPU infrastructure—only API access. This would democratize agent training, extending access to independent developers, open-source communities, and smaller institutions currently priced out of the field.

Lower-cost agent optimization would generate substantial productivity gains across knowledge sectors. Organizations could rapidly develop domain-specific agents for complex, multi-stage workflows in legal analysis, scientific research, engineering design, and financial modeling. Because these agents could be iteratively improved without retraining model weights, firms could deploy specialized automation at a fraction of current costs. At the industry level, this would accelerate the transition from generic AI assistants to customized systems embedded in workflows—enabling automation of multi-step planning and tool-use sequences across logistics, healthcare, manufacturing, and customer operations.

More broadly, reducing dependence on large-scale compute would weaken the concentration of AI capabilities within major labs. Today, the high fixed costs of RLHF and agent training reinforce the dominance of organizations with access to large GPU clusters. A low-cost alternative would allow open-source communities and smaller institutions to compete on capability rather than capital, potentially leading to a more diverse and competitive AI ecosystem. The scientific sector would benefit as well: affordable trajectory-aware optimization could accelerate progress in fields like drug discovery, materials science, and climate modeling by enabling rapid iteration and domain-specific agent customization without specialized RL infrastructure.

1.8.2 Technical Contribution

This project will, to our knowledge, produce the first open-source (MIT-licensed) library bridging GEM and DSPy, enabling future research into hybrid optimization methods that combine evolutionary prompts with lightweight fine-tuning.

Chapter 2

Literature Review

In the extensive and rapidly expanding literature on LLM-based agents, we focus on the following key themes that are most relevant for this study:

2.1 Reinforcement Learning Methods and Gym Environments . . .	11
2.2 Prompt Engineering and Evolutionary Methods	14
2.3 Bridging Gyms and Prompt Optimization	17
2.4 Literature Review Conclusion	20

Section 2.1 presents the weight-space paradigm, covering reinforcement learning methods and the agentic gym environments that enable their training and evaluation. Section 2.2 explores token-space and prompt optimization as an alternative to weight-space methods. Finally, Section 2.3 synthesizes these approaches, bridging gym environments with prompt optimization techniques.

2.1 Reinforcement Learning Methods and Gym Environments

The canonical way of improving neural network (and thus LLM) performance is optimizing the parameter weights of the underlying model. With the explosion of interest in LLMs since the release of OpenAI’s GPT-3 model, there is now a large and growing body of research evaluating how LLM agents perform not just at simple queries but at complex tasks, where success depends on multi-step decisions such as planning, tool use, and recovery from intermediate errors. In these tasks, performance can degrade through repeated action loops, shallow tool exploration, and inability to generalize the training set (X. Liu et al., 2024; Qin et al., 2023). This makes it critical to evaluate agents in a standardized environment (e.g., agentic “gyms”) using a rigorous, end-to-end framework that has visibility of each decision step (e.g. reinforcement learning with a well-specified reward and update policy).

2.1.1 Foundations of RL for LLM Agents

While RL methods have a long history of being applied in machine learning, one of the key foundations of RL for agents is the Trust Region Policy Optimization (TRPO) framework, introduced by Schulman et al. (2015), which constrains policy updates to improve stability. Schulman et al. (2017) then provided a practical approximation to TRPO with Proximal Policy Optimization (PPO), a clipped-objective alternative that has become widely used. The PPO-clip objective is expressed as

$$L_{\theta}^{CLIP}(\theta) = \mathbb{E}_t[\min r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t] \quad (2.1)$$

1. θ are the policy parameters
2. $r_t = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$, the probability ratio. It measures how much the policy changed for a given state-action pair.
3. $\hat{A}_t$ is estimated advantage. It is large and positive for good actions.
4. ϵ is the clipping hyperparameter that controls how much the policy can change per update

Using a clipping mechanism on the probability ratio ensures that changes to the model remain small. Overall, TRPO and PPO form common baselines for sequential decision-making, providing stabilization mechanisms through trust-region constraints and clipped objectives, respectively.

2.1.2 Preference-Based and Outcome-Driven Methods

Building on these foundational policy optimization methods, subsequent research has focused on aligning agent behavior with human preferences. Once the policy framework has been set, another key challenge in agentic RL is specifying the right rewards to achieve the intended outcomes. Christiano et al. (2017) provided an early and important contribution in this area when they showed that learned rewards from pairwise comparisons by humans can guide policy learning when explicit rewards are unavailable. Ziegler et al. (2019) then applied preference-based fine-tuning to language models, showing how learned rewards and KL-style constraints can shape model behavior while preserving fluency. Stiennon et al. (2020) demonstrated a related pipeline for summarization, optimizing toward human-preferred summaries rather than overlap-based metrics.

Ouyang et al. (2022) extended this approach for instruction following in InstructGPT and reported substantial gains in alignment with user intent. Bai et al. (2022) proposed Constitutional AI, using principle-guided AI feedback to scale alignment signals and reduce reliance on direct human annotation. Rafailov et al. (2023) introduced Direct Preference Optimization (DPO), reframing preference learning so behavior can shift without a separate reinforcement learning loop, while still training directly on preference comparisons.

While preference-based optimization has proved effective for aligning language models with human judgment, it remains computationally expensive and reliant on complex reward modeling pipelines. Recent work by Shao et al. (2024) and DeepSeek-AI addresses these

limitations by shifting toward outcome-driven reinforcement learning paradigms that prioritize computational efficiency while maintaining similar performance. In *DeepSeekMath*, Shao et al. (2024) introduced Group Relative Policy Optimization (GRPO), a method designed to obviate the need for a memory-intensive value function (critic). Instead of training a separate critic to estimate baselines, GRPO samples a group of trajectories for a single input and computes the advantage of each trajectory relative to the group’s mean reward. This group-based formulation significantly reduces training resource requirements and stabilizes optimization by normalizing advantages against the group distribution. DeepSeek-AI further validated this approach in *DeepSeek-R1*, demonstrating that GRPO—when combined with verifiable outcome rewards—can incentivize the spontaneous emergence of self-verification and reasoning refinement behaviors without relying on explicit process supervision (DeepSeek-AI, 2025).

2.1.3 Credit Assignment in Multi-Turn Settings

While preference-based methods address *what* outcomes to optimize for, a parallel challenge is determining *which* decisions contributed to those outcomes. Once the optimization framework is in place, the final key component of a multi-turn RL-based agentic system is credit assignment. As Sutton and Barto (2018) showed, this becomes more complicated as task complexity grows. Pignatelli et al. (2023) surveyed temporal credit assignment in deep reinforcement learning and emphasized how delayed and noisy feedback complicates learning dynamics across methods intended to propagate credit through time. In the specific context of mathematical reasoning, Lightman et al. (2023) demonstrated that process supervision, rewarding correct reasoning steps rather than just the final outcome, substantially improves performance compared to outcome supervision. However, as Z. Liu et al. (2025) observe in GEM, GRPO inherently assigns a single trajectory-level advantage to all tokens in a generated sequence. This coarse-grained credit assignment becomes a limitation in complex multi-turn agentic tasks, where distinguishing the specific contributions of intermediate decisions is critical.

2.1.4 Gym Environments for Agentic Evaluation

One of the foundational underpinnings of agentic gyms comes from Ng et al. (1999), who showed that intermediate reward signals can accelerate learning while preserving optimal policies under specific conditions. This made it clear that environments that could provide this kind of dense feedback would be critical to improving the performance of RL systems.

In 2016, Brockman et al. (2016) proposed the first major implementation of this kind of environment with the OpenAI Gym. This gave developers building large-scale RL systems a consistent evaluation framework through a shared API for observations, actions, and rewards. Over the past decade, there have been a number of other gym frameworks that have sought to improve on Brockman’s initial implementation, most recently and relevantly from Z. Liu et al. (2025), who proposed GEM as a gym specifically tailored for complex agentic LLMs, emphasizing standardized rollouts, per-step transition logging, and per-turn reward collection to support both evaluation and training.

When it comes to using these gyms to train and deploy multi-turn agentic systems, three challenges recur in the literature. First, multi-turn settings amplify early mistakes because small misinterpretations can propagate across an episode (X. Liu et al., 2024). Second, many agent environments suffer from limited observability; Kaelbling et al. (1998) formalized planning and acting under partial observability and emphasized that robust policies must cope with uncertainty and incomplete state information. Wei et al. (2022)—and many others since—have shown that chain-of-thought prompting can compensate for this, thereby improving multi-turn task performance. Third, evaluation can be noisy and expensive due to random variation and non-determinism in external tools.

X. Liu et al. (2024) introduced AgentBench to evaluate LLMs as agents across multiple interactive environments and highlighted that long-horizon reasoning and instruction adherence remain common failure points. S. Zhou et al. (2023) introduced WebArena, focusing evaluation on realistic web interaction and long-horizon tasks. Jimenez et al. (2024) introduced SWE-bench for GitHub issues where agents must iteratively diagnose, patch, and validate solutions. Together, these benchmarks reinforce the need to evaluate agents using both outcome metrics and behavioral measures such as efficiency, stability, and tool-use correctness. When comparing optimization methods with fundamentally different cost structures, such as upfront RL training versus per-inference prompt overhead, equivalence testing frameworks like Two One-Sided Tests (TOST) provide a principled statistical approach for establishing comparable rather than merely superior performance (Lakens, 2017).

2.2 Prompt Engineering and Evolutionary Methods

Beyond optimizing model weights through RL, token-space methods offer a complementary way to improve agentic task performance. Rather than updating parameters through gradient-based learning, token-space optimization refines the instructions, demonstrations, reasoning scaffolds, and tool-use protocols provided within the LLM context window. This approach is appealing because it is model-agnostic, enables rapid iteration without access to model internals, and avoids the computational overhead of fine-tuning. In practice, task decomposition, memory scaffolding, and tool-use instructions can be modified and productionized quickly with minimal infrastructure.

2.2.1 Structured Reasoning as a Token-Space Foundation

Before examining automated prompt optimization, it is important to establish why token-space design matters for complex reasoning tasks. Wei et al. (2022) introduced chain-of-thought (CoT) prompting, showing that providing intermediate reasoning steps in few-shot exemplars can substantially improve performance on arithmetic, commonsense, and symbolic reasoning tasks. Their experiments demonstrated that these reasoning behaviors emerge more strongly at larger model scales, exhibiting dramatically increasing scaling curves on tasks where standard prompting shows relatively flat performance. This finding established a foundational insight: the structure of the prompt—not merely its semantic content—can unlock latent reasoning capabilities in large language models.

Yao, Yu, et al. (2023) extended this line of work with Tree of Thoughts (ToT), proposing

deliberate search over intermediate "thoughts" rather than a single linear chain. Unlike CoT which follows a single reasoning trajectory, ToT maintains multiple candidate reasoning paths and uses search strategies—such as breadth-first or depth-first exploration with model-based evaluation—to select promising continuations and allow backtracking when necessary. The key innovation is enabling LMs to perform deliberate decision-making by considering multiple different reasoning paths and self-evaluation choices. On challenging tasks like Game of 24, ToT reports substantial improvements over strong CoT baselines under comparable evaluation settings.

Taken together, CoT and ToT show that token-space scaffolding can be treated as an explicit mechanism for shaping intermediate computation: CoT makes reasoning steps part of the prompt, and ToT turns multi-step reasoning into a search procedure at inference time (Wei et al., 2022; Yao, Yu, et al., 2023). This suggests a natural next step: if structured prompting and search over reasoning traces can improve performance within a single problem, then the prompt structures that induce such behaviors can themselves be treated as objects of optimization across tasks, with performance serving as the selection signal.

2.2.2 Automated Prompt Optimization

Given the demonstrated importance of prompt structure, a natural question arises: can prompt design be automated? Early work showed that the answer is affirmative, though the methods vary considerably in their assumptions and mechanisms.

Y. Zhou et al. (2022) proposed the Automatic Prompt Engineer (APE), treating instruction induction as a black-box optimization problem. APE generates candidate instructions from a small set of input-output demonstrations, evaluates them using a target model, and iteratively improves high-performing candidates via Monte Carlo search that proposes semantically similar instruction variants. Concurrently, Madaan et al. (2023) introduced Self-Refine, demonstrating that a single LLM can iteratively enhance its outputs through intrinsic feedback loops—self-generated critiques followed by revisions—without requiring supervised data, fine-tuning, or reinforcement learning. While APE optimizes prompts and Self-Refine optimizes outputs, both methods establish a common principle: LLMs can function as both generator and improver within iterative optimization loops.

Khattab et al. (2024) introduced DSPy, which treats LLM pipelines as modular programs whose structure (modules and control flow) is specified in code, while prompting-related choices (instructions, few-shot demonstrations, and LM configurations) are treated as tunable parameters optimized against explicit metrics. DSPy’s optimizers (teleprompters) "compile" a program by automatically selecting or generating effective prompts and demonstrations on a small development set under a user-defined evaluator, reducing prompt engineering from manual string editing to a reproducible optimization workflow. This often yields significant quality gains without updating the model weights, and the same program can be recompiled whenever the implementation, data, or metrics change.

Collectively, these methods recast prompting from manual string crafting into a search problem: the prompt becomes a set of tunable variables, and performance is improved through iterative propose–evaluate–update loops (Madaan et al., 2023; Y. Zhou et al., 2022). However, much of this automation still relies on relatively local or small-batch exploration of a vast discrete search space, motivating broader search strategies that maintain diversity and

explore multiple competing hypotheses in parallel, a role naturally filled by population-based evolutionary algorithms.

2.2.3 Evolutionary Prompt Optimization

Evolutionary algorithms provide a principled framework for this search problem. By maintaining a diverse population of candidate prompts and applying selection operations such as crossover and elitism over generations, these methods can explore the prompt space more thoroughly while avoiding local optima.

A critical contribution in this area comes from Guo et al. (2024), who proposed EvoPrompt, connecting large language models with classical evolutionary algorithms to optimize prompts without gradient access. The key innovation is using the LLM itself as the selection operator. Instead of having fixed operators, like crossover, using the LLM as the selection operator allows one to parameterize a selection operator via the LLM weights. This allows for flexible and different selection operators. EvoPrompt instantiates two widely-used algorithms: Genetic Algorithm (GA), where LLMs perform crossover on two parent prompts and then mutation on the resulting offspring; and Differential Evolution (DE), which emphasizes mutating only the differing parts between prompts while preserving shared components, then recombining segments to create new candidates. A fitness function evaluates each candidate on a held-out development set, and selection pressure drives the population toward higher-performing prompts over successive generations. EvoPrompt demonstrates significant improvements over human-engineered prompts and existing automatic prompt generation methods across language understanding, generation, and reasoning tasks.

Fernando et al. (2023) introduced PromptBreeder, extending evolutionary prompt optimization with a self-referential mechanism. The central innovation is that PromptBreeder evolves not only task-prompts but also mutation-prompts—the instructions that specify how task-prompts should be modified. This *improving the improver* design moves beyond evolving a single prompt population toward evolving the update operators themselves. PromptBreeder employs multiple classes of mutation operators, including zero-order generation (producing hints from problem descriptions), first-order generation (applying mutation-prompts to task-prompts), and hyper-mutation (mutating the mutation-prompts themselves). The approach demonstrates that automated evolution can discover effective and sometimes un-intuitive prompts that outperform hand-crafted strategies.

Together, these methods create a continuum of optimization approaches: from internal self-correction (Self-Refine), to programmatic compilation (DSPy), to generate-and-select search (APE), and finally to population-based evolution (EvoPrompt, PromptBreeder) (Fernando et al., 2023; Guo et al., 2024; Khattab et al., 2024; Madaan et al., 2023; Y. Zhou et al., 2022). Each step increases the sophistication of the search process while remaining entirely within the token space, providing direct precedents for comparing token-space methods against weight-space reinforcement learning approaches.

2.2.4 Limitations for Multi-Turn Agentic Tasks

Despite rapid progress, prompt optimization methods share limitations that become more pronounced in multi-turn agentic settings.

First, most prompt optimization research focuses on single-turn tasks where fitness depends on a single model output. Multi-turn agents, by contrast, unfold over trajectories where early errors compound and intermediate decisions determine long-horizon outcomes. The fitness landscape for such tasks is fundamentally different: a prompt that performs well on isolated turns may lead to poor trajectory-level outcomes due to compounding errors or suboptimal action sequences.

Second, optimization signals are often coarse-grained. When only terminal success or failure is scored, it becomes difficult to attribute outcomes to particular prompt components. This echoes the broader credit assignment challenge in sequential decision-making: delayed rewards create ambiguity about which decisions deserve credit, and this challenge intensifies with longer horizons (Sutton & Barto, 2018).

Third, prompt evolution has seen relatively limited integration with gym-based evaluation infrastructure. Agentic environments expose per-turn transitions, rewards, and trajectory logs that could provide dense feedback signals for guiding prompt optimization, but this connection remains underexplored.

Fourth, generalization remains an open concern: optimized prompts risk overfitting to the specific task variations seen during optimization, potentially failing to transfer to held-out examples or related tasks. Programmatic frameworks like DSPy mitigate this through modular decomposition and recompilation (Khattab et al., 2024), while Pareto-aware methods like GEPA maintain population diversity to avoid convergence on narrow solutions (Agrawal et al., 2025), but systematic study of prompt transferability in agentic settings is limited.

Finally, real deployments often involve multiple competing objectives—correctness, cost, latency, stability, and safe tool use—that cannot be collapsed into a single scalar fitness. Multi-objective optimization frameworks exist for approximating Pareto fronts across conflicting criteria (Deb et al., 2002), but their integration with prompt evolution for agentic tasks remains an open area.

These gaps motivate the trajectory-aware approach discussed in Section 2.3: combining token-space evolution with dense, per-turn signals from agentic gyms to guide prompt optimization for multi-turn tasks.

2.3 Bridging Gyms and Prompt Optimization

We now shift to the more nascent literature on trajectory-aware prompt evolution. The central premise is that per-turn rewards from an agentic gym can be repurposed as a fitness function. Specifically, the method of calculating a total trajectory return from per-turn rewards is the fitness function, denoted as F . The total return is the fitness value, a scalar.

$$F : S \rightarrow \mathbb{R} \tag{2.2}$$

The domain $S \subset \mathbb{R}^n$, where n is to be determined. If we decide to embed the prompts as vectors, then n is the dimension of the embedding. If we decide to keep the prompts as strings, then n is the length of the context window. With search and evolutionary methods, we can optimize a prompt without gradients.

2.3.1 Credit Assignment in Token Space

Credit assignment is an obstacle for long-horizon optimization in both reinforcement learning and prompt-based control. In classical RL, delayed rewards in a Markov decision process make it difficult to determine which actions contributed to eventual success. This ambiguity grows with horizon length (Sutton & Barto, 2018). Modern LLM-training pipelines inherit this difficulty: PPO stabilizes KL-regularized policy updates in RLHF systems (Schulman et al., 2017), while DPO reframes preference optimization as a supervised objective without an explicit reward model (Rafailov et al., 2023). GRPO extends preference-based optimization with improved sample efficiency (Ramesh et al., 2024). Yet, all of these methods still rely on attributing global sequence-level signals to local token-level decisions.

Dense reward strategies and step-by-step verification attempt to mitigate this obstacle by providing intermediate correctness signals, but the mapping from individual tokens to downstream performance remains highly nonlinear. This mirrors earlier sequence-level RL work in NLP, where sparse sequence-level rewards produce high-variance gradients and unstable learning (Bahdanau et al., 2017; Ranzato et al., 2016). CoT prompting further illustrates the challenge: small perturbations to early reasoning steps can cascade into qualitatively different trajectories (Wei et al., 2022), making it difficult to isolate which prompt components improve or degrade performance.

OpenAI Gym establishes standardized RL benchmarks with long-horizon, sparse reward tasks (Brockman et al., 2016). Newer LLM-centric environments such as GEM extend this paradigm to agentic systems, where prompts act as compact policies executed all at once. Evolutionary prompt optimization methods such as Self-Refine, APE, EvoPrompt, and PromptBreeder attempt to bypass gradient-based credit assignment by evaluating whole-prompt variants. However, they still face non-separable fitness landscapes, where small token mutations have unpredictable effects (Salimans et al., 2017). Programmatic prompting frameworks like DSPy introduce modular, verifiable prompt components (Khatab et al., 2024), but even these systems must infer which module or token contributes to success when only terminal outcomes are observable.

2.3.2 Multi-Objective Optimization

A second challenge is objective mismatch, which means minimizing the objective function fails to create intended agentic behavior. Agents are typically evaluated on multiple criteria: correctness, cost, stability, and safe tool use. Deb et al. (2002) introduced NSGA-II, a widely used algorithm for approximating Pareto fronts across multiple objective functions. Recent prompt evolution work connects directly to this framing. Agrawal et al. (2025) introduced GEPA, using reflective prompt evolution with Pareto-aware selection to improve compound AI systems against evaluation metrics. GEPA provides a directly relevant baseline conceptually aligned with trajectory-aware prompt evolution.

Gym-style agentic environments further highlight the need for multi-objective evaluation. OpenAI Gym tasks often require balancing reward, stability, and constraint satisfaction (Brockman et al., 2016). LLM-based environments such as GEM extend this to tool-augmented agents whose prompts must jointly optimize accuracy, safety, and resource usage (Z. Liu et al., 2025). The evolutionary prompt optimization methods naturally maintain

diverse populations of prompts and are thus well suited for exploring Pareto-front trade-offs across multiple objective functions.

Prompting research reinforces this need. CoT prompting improves reasoning quality (Wei et al., 2022) but can increase latency or verbosity. Programmatic prompting frameworks like DSPy (Khatab et al., 2024) allow explicit modularization of objectives but still require multi-metric evaluation to avoid regressions in safety or cost. As agents become embedded in real workflows, optimizing a single scalar reward becomes insufficient; multi-objective methods become essential, whether the methods are RL-based, evolutionary, or programmatic.

2.3.3 Dense Feedback Mechanisms

Dense feedback mechanisms, which provide evaluative signals at every possible step rather than only at the end of a trajectory, emerge in the literature as one of the most reliable ways to mitigate sparse reward pathologies. Potential-based reward shaping formalizes how intermediate rewards can accelerate learning without altering the optimal policy (Ng et al., 1999). This principle directly anticipates the challenges faced in token-level optimization, where long-horizon dependencies make sparse terminal rewards especially uninformative.

Contemporary agentic environments adopt this principle extensively. GEM provides step-wise evaluative signals for LLM-driven agents and enables more stable optimization in long-horizon tasks (Z. Liu et al., 2025). By providing feedback at each interaction step rather than only after a full reasoning trajectory or tool-use sequence, these environments reduce credit assignment ambiguity and improve the sample efficiency of both RL-based and evolutionary optimization methods.

Dense feedback also aligns with trends in LLM training and prompting. Step-by-step verification methods provide intermediate checks on correctness during reasoning and effectively transform sparse sequence-level evaluation into a denser reward landscape (Lightman et al., 2023; Wei et al., 2022). Programmatic prompting frameworks similarly encourage modular decomposition of tasks and allow evaluators to attach feedback to individual sub-components rather than entire outputs (Khatab et al., 2024). Evolutionary prompt optimization methods benefit as well: when each mutation can be assessed on multiple intermediate criteria, search becomes more directed and less reliant on noisy terminal outcomes.

2.3.4 Structured Agent Patterns

Structured prompting patterns provide another useful paradigm for implementing prompt optimization in agentic gyms, particularly for improving multi-step tool-grounded behavior. Yao, Zhao, et al. (2023) introduced ReAct, which interleaves reasoning and action in a way that improves task execution in interactive settings and supports clearer diagnosis of intermediate decisions. Schick et al. (2023) introduced Toolformer and showed that models can learn when to call tools and how to incorporate tool outputs, reinforcing that tool interaction is a core capability for agentic evaluation. Qin et al. (2023) introduced ToolLLM, including the ToolBench evaluation, to measure tool-use behavior across a large set of real APIs and highlight failure patterns such as incorrect argument selection and shallow tool exploration.

2.4 Literature Review Conclusion

The literature collectively points to token-space optimization within agentic gym environments as a compelling direction for advancing the capabilities of LLM-driven agents. Across reinforcement learning, agentic gyms, evolutionary search, and modern prompting frameworks, a consistent theme emerges: long-horizon decision-making demands mechanisms that expose richer structure, denser evaluative signals, and more interpretable intermediate behaviors. Gym-based evaluation infrastructures through systematic logging, intermediate feedback, and domain-specific benchmarks provide the scaffolding needed to study these dynamics in a controlled yet extensible setting, which makes them natural testbeds for prompt-level and policy-level optimization.

At the same time, developments in structured reasoning, programmatic prompting, and reflective self-improvement highlight that prompts can function as modular, revisable policies rather than static text strings. These approaches reveal that multi-step reasoning quality, stability, and safety can all be improved when prompts are optimized with awareness of their downstream trajectories. Evolutionary methods, dense-reward RL techniques, and step-wise verification strategies each contribute complementary tools for navigating the complex, multi-objective landscapes that arise in real deployments.

On the whole, the literature suggests that integrating dense feedback, multi-objective evaluation, and trajectory-aware prompt evolution offers a promising path toward more reliable, adaptable, and high-performing agentic systems. As agentic workflows grow more complex, these combined insights provide a strong foundation for further investigation into scalable, interpretable, and robust token-space optimization. The following chapter details our methodology for developing and empirically testing this integration.

Chapter 3

Methodology

3.1 Research Design

This study employs an experimental design to evaluate prompt optimization against reinforcement learning for agentic LLM tasks. The independent variable is the optimization method: GEPA-based prompt optimization versus RL baselines (REINFORCE with Return Batch Normalization, PPO, and GRPO) as reported in the GEM framework (Z. Liu et al., 2025). The dependent variables are task success rate, computational cost, and trajectory efficiency.

3.1.1 Comparative Framework

The experimental design centers on a controlled comparison where both optimization paradigms are evaluated on identical task environments using identical model parameters, since each LLM’s weights are frozen. We use the GEM gym framework as our evaluation testbed because it provides: (1) standardized environments spanning mathematical reasoning, code generation, question answering, and terminal-based tool use; (2) per-turn dense reward signals enabling fine-grained credit assignment; and (3) published RL baselines against which prompt-optimized agents can be directly compared.

Rather than training RL agents ourselves, which would require substantial GPU infrastructure and introduce implementation variance, we compare GEPA-optimized agents against the published baselines from the GEM paper. This approach isolates the optimization method as the independent variable while controlling for environment and evaluation methodology.

3.1.2 Gym Environment Selection

We surveyed major agentic training environments including AgentGym-RL (Xi et al., 2025), LMRL-Gym (Marber et al., 2024), MLGym (AI, 2025), AgentBench (X. Liu et al., 2024), and NeMo Gym (NVIDIA, 2025). We selected **GEM** for three reasons: (1) its explicit support for *per-turn dense rewards* enables the fine-grained credit assignment central to our trajectory-aware fitness design; (2) its standardized OpenAI Gym interface (`reset`, `step`) provides the architectural foundation for our GEM-DSPy adapter; and (3) its environments

(Math12K, CodeContest, HotpotQA) represent reasoning-intensive tasks where the RL vs. prompt optimization comparison is most relevant.

3.1.3 Experimental Conditions

Test (GEPA Prompt Optimization): Agents are implemented as DSPy modules with frozen LLM weights. GEPA evolves the system prompts, instructions, and reasoning scaffolds through reflective mutation, using GEM environment rewards as the fitness signal. Optimization occurs entirely in token space with no gradient updates to model parameters.

Control (RL Baselines): Published results from GEM using REINFORCE+ReBN, PPO, and GRPO on Qwen3 models (1.7B, 4B parameters). These baselines represent the weight-space optimization paradigm where model parameters are updated via policy gradients.

3.1.4 Task Environments and Evaluation Protocols

We evaluate across three GEM environments: Math12K (chain-of-thought mathematical reasoning), CodeContest (competitive programming with automated test verification), and HotpotQA (multi-hop question answering). These span single-turn reasoning, tool-augmented code execution, and knowledge-intensive retrieval.

To ensure valid comparison, we adopt GEM’s evaluation protocols: the agent uses a maximum response length of 4096 tokens per turn, sampling temperature $T = 1.0$ during training and $T = 0.0$ during evaluation, with top- $p = 1.0$ and top- k disabled. The latter are two disabled to ensure the model’s full vocabulary is available. The temperature is set to 0 during evaluation to ensure no exploration. The model simply chooses the most probable next token. Agents interact through identical observation-action interfaces with matched tool access, and we use GEM’s designated held-out test sets.

3.2 Technical Components

The implementation consists of four integrated components: (1) a GEM-DSPy adapter that bridges gym environments with prompt optimization, (2) trajectory-aware fitness functions that convert per-turn rewards into GEPA-compatible metrics, (3) the GEPA optimizer configured for multi-turn agentic tasks, and (4) AWS-based LLM infrastructure for both task execution and reflective optimization. Expanding on equation (2.2), the one possible fitness function is

$$F(\tau) = \sum_{t=0}^T \gamma^{T-t} \cdot w_t \cdot R_T + \lambda \sum_{t=0}^T r_t \quad (3.1)$$

τ is the vector of trajectories: $(s_0, a_0, s_1, \dots, s_T)$. R_T is the final reward indicator, 1 for correct answer and 0 for wrong. r_t are per-turn auxiliary rewards, like syntactic checks. γ is the discount factor, notice that the weighting is reverse in time. This means that final step receives full credit, while those earlier in the trajectory receive much less credit. w_t weights the contribution of turn.

The GEPA optimizer works under expectation maximization:

$$\max_{\Omega} \mathbb{E}_{\tau \sim p_{\Omega}(\tau)}[F(\tau)] \quad (3.2)$$

It finds the best set of prompt tokens in Ω . The formula tells us that for a given set of prompt tokens Ω , we generate trajectories τ according to the probability distribution $p_{\Omega}(\tau)$, and we want to maximize the expected fitness $F(\tau)$ across all possible trajectories.

3.2.1 GEM-DSPy Adapter

The adapter translates between GEM’s OpenAI Gym interface (`reset`, `step`, `observation`) and DSPy’s module abstraction. This translation layer serves as the “connective tissue” enabling prompt-optimized agents to operate within RL-designed environments.

Environment Wrapping: Each GEM environment is wrapped to expose a consistent interface for DSPy agents. The wrapper handles:

- Observation formatting: converting environment state to text-based observations suitable for LLM consumption.
- Action extraction: parsing LLM outputs into valid environment actions.
- Trajectory logging: capturing the full (s_t, a_t, r_{t+1}) sequence for fitness computation and GEPA reflection.
- Tool integration: routing tool calls (Python execution, web search, shell commands) through GEM’s standardized interfaces.

Agent Architecture: Agents are implemented as DSPy modules following the ReAct pattern (Yao, Zhao, et al., 2023):

- `dspy.ChainOfThought`: For single-turn reasoning tasks (Math12K).
- `dspy.ReAct`: For multi-turn tool-using tasks (Terminal, CodeContest) with interleaved reasoning and action.

The choice of module determines which prompt components GEPA can optimize: ChainOfThought exposes the reasoning instruction, while ReAct additionally exposes tool descriptions that can be jointly evolved when `enable_tool_optimization=True`.

3.2.2 GEPA Optimizer Configuration

GEPA operates through reflective prompt evolution: an LLM analyzes execution traces and proposes prompt modifications that address observed failure modes. We configure GEPA with the following key parameters:

Reflection Model: Claude 4.5 Sonnet with high temperature ($T = 1.0$) for diverse mutation proposals. The reflection model is distinct from the task model to avoid conflating optimization signal with task execution. Task model refers to the model performing agentic tasks.

Pareto-Aware Selection: Rather than optimizing a single “best” prompt, GEPA maintains a Pareto frontier of prompts that each excel on different subsets of the training tasks. A Pareto frontier is the set of all Pareto optimal solutions.

$$\mathcal{P} = \{x^* \in X : x^* \text{ is Pareto Optimal}\}$$

A solution x^* is Pareto optimal if no other solution dominates it. In our case, this means the prompt has the highest fitness score out of all other prompts for a given task. This prevents premature convergence and preserves diverse strategies that may generalize differently to test tasks.

Budget Configuration: We use GEPA’s `auto='medium'` budget for primary experiments, which balances optimization thoroughness against API costs. Ablations with `'light'` and `'heavy'` budgets quantify the cost-performance tradeoff.

Composite Fitness Terms: Beyond task success, we incorporate behavioral objectives:

- **Loop detection penalty:** Reduces fitness for trajectories exhibiting repeated identical actions.
- **Step efficiency bonus:** Rewards shorter successful trajectories (aligned with $\gamma < 1$ in the return formulation).

3.2.3 AWS Infrastructure

All LLM inference is conducted via AWS to ensure reproducibility and controlled cost measurement.

LiteLLM Integration: DSPy routes all model calls through LiteLLM as a unified provider layer. This normalizes configuration across Amazon Bedrock endpoints (model IDs, regions, and auth), enables consistent retry/backoff behavior, and centralizes request logging for cost and latency tracking.

Task Models (Agent Execution): We use Amazon Bedrock to serve Qwen3 models (matching GEM’s RL baselines) as well as Claude models (generalization experiments). Using the same model family as GEM’s RL experiments ensures the comparison isolates optimization method rather than model capability.

Reflection Models (GEPA Mutation): Amazon Bedrock serves Claude 4.5 Sonnet for reflective prompt evolution. The reflection model requires strong instruction-following and reasoning capabilities to analyze trajectories and propose effective mutations.

Cost Tracking: All API calls are logged with token counts and latency, enabling precise compute cost comparison between GEPA optimization (inference-only) and RL training (as reported in GEM).

Token Budget Estimate: Based on GEPA’s medium budget configuration (75 iterations, 6 candidates, 3 tasks per minibatch, 3 replications), we estimate tokens per model as follows.

Define a study as a set of experiments. Let R be the number of replications. For example, if $R = 3$, then the experiment is run 3 times for a given study, namely:

$$S = e_1, e_2, e_3, \quad |S| = R = 3.$$

The total tokens in a study is given by

$$T = R \cdot (t_{train} + t_{val} + t_{eval} + t_{ref}). \quad (3.3)$$

t is the number of tokens a each stage of a the modeling process. The subscript denotes the step in the modeling process, training, validation, evaluation, and reflection, respectively. The formula for the number of tokens used is training is

$$t_{train} = C \cdot I \cdot M \cdot E_{train} \cdot \bar{t}$$

C is the number of candidates, I is number of iterations (number of gradient updates), M is the number of tasks per iteration, E_{train} is the number of episodes per task, where an episode is a single attempt at a task, and $\bar{t}$ is the number of tokens per episode. By GEPA’s configuration: $C = 6$ candidate, $I = 75$ iterations, $M = 3 \frac{tasks}{iteration}$, $E_{train} = 10 \frac{episodes}{task}$, and $\bar{t} = 2000 \frac{tokens}{episode}$. This gives the total tokens used in training per experiment:

$$t_{train} = 6 \cdot 75 \cdot 3 \cdot 10 \cdot 2000 = 2.7 \times 10^7 \text{ tokens}$$

Validation tokens are similar but evaluation is only done periodically and we validate on other environments. The formula for total tokens used in one experiment in the validation stage is

$$t_{val} = C \cdot \frac{I}{V_{freq}} \cdot total_{env} \cdot E_{val} \cdot \bar{t}$$

We have $C = 6$ candidates, $I = 75$ iterations, $V_{freq} = 5$ (validate every 5 iterations), $total_{env} = 5$ environments, $E_{val} = 3 \frac{episodes}{env}$ (shorter than training), $\bar{t} = 2000 \frac{tokens}{episode}$. So then,

$$t_{val} = 6 \cdot 75/5 \cdot 5 \cdot 3 \cdot 2000 = 2.7 \times 10^7 \text{ tokens}$$

Now for t_{eval} , which is essentially evaluation on the test set.

$$t_{eval} = C \cdot total_{env_tests} \cdot E_{eval} \cdot \bar{t}$$

Technically $\bar{t}$ should be less during testing because we are not doing exploration in the test phase but for simplicity we keep it the same. We have $C = 6$ candidates, $total_{env_tests} = 5$

unseen environments, $E_{eval} = 20$ episodes per environment (for statistical significance), $\bar{t} = 2000 \frac{\text{tokens}}{\text{episode}}$. This gives us

$$t_{eval} = 6 \cdot 5 \cdot 20 \cdot 2000 = 1.2 \times 10^6 \text{ tokens}$$

Finally, we have total tokens used in an experiment during reflection:

$$t_{ref} = C \cdot \frac{I}{R_{freq}} \cdot (L_{analysis} + L_{summary})$$

$C = 6$ candidates, $I = 75$ iterations, $R_{freq} = 5$ (reflect every 5 iterations), $L_{analysis} = 5000$ tokens (on average), and $L_{summary} = 1000$ tokens (on average). This results in

$$t_{ref} = 6 \cdot 75/5 \cdot (5000 + 1000) = 5.4 \times 10^5 \text{ tokens}$$

Plugging these values into (3.1), we get

$$T = 3 \cdot (3.144 \times 10^7) = 9.432 \times 10^7 \text{ tokens}$$

Pricing per 1K tokens (AWS Bedrock): Qwen3 \$0.00015 input/\$0.0006 output; Claude Sonnet 4.5 \$0.003 input/\$0.015 output; GPT-4o \$0.005 input/\$0.015 output. Note that GPT-4o isn’t available through AWS Bedrock, but it can be accessed via an external API.

Component	Input Cost	Output Cost	Total Cost
Qwen3.1	\$11.32	\$11.32	\$22.64
GPT-4o	\$377.28	\$282.96	\$660.24
Claude Sonnet	\$226.37	\$282.96	\$509.33
Total	\$614.87	\$577.24	\$1192.11
With 25% buffer	\$768.59	\$721.55	\$1490.14

To accommodate stretch goals, an ideal budget of approximately \$2,500 would provide sufficient headroom.

3.3 Metrics and Model Evaluations

We evaluate performance across three dimensions: task effectiveness, computational efficiency, and behavioral quality. These metrics directly map to our hypotheses (H1: performance parity, H2: compute efficiency, H3: behavioral mechanisms).

3.3.1 Primary Metrics

Task Success Rate: The primary performance metric is task success rate—the proportion of test episodes where the agent achieves the task objective. This metric is directly comparable to GEM’s published RL baselines and serves as the basis for hypothesis H1. For mathematical reasoning tasks, success requires producing the correct final answer. For code generation, success requires passing all test cases. For terminal/Docker tasks, success requires achieving the specified system state. For question-answering, success requires matching the ground-truth answer (evaluated via exact match or semantic equivalence depending on the dataset).

Statistical Testing: We use Two One-Sided Tests (TOST) to assess equivalence between GEPA and RL baselines at $\alpha = 0.05$ with an equivalence margin of ± 5 percentage points. This approach is appropriate because our hypothesis is that GEPA achieves *comparable* (not superior) performance; traditional hypothesis testing would not distinguish “no detectable difference” from “insufficient power to detect a difference.”

For each environment, we report:

- Point estimates with 95% confidence intervals (computed via bootstrap resampling with 1,000 iterations).
- TOST p-values for equivalence claims.
- Effect sizes (Cohen’s d) to characterize practical significance.

3.3.2 Secondary Metrics

Compute Cost: Measured in two complementary units: wall-clock time (total optimization duration) and dollar cost (estimated monetary cost based on AWS pricing). For GEPA, cost accumulates through API calls during optimization iterations. For RL baselines, we use the training costs reported in the GEM paper: all GEM experiments were conducted on $8 \times$ A100 GPUs and completed in approximately one day (Z. Liu et al., 2025).

Trajectory Efficiency: Mean number of steps to successful task completion. Lower values indicate more efficient agent behavior. This metric is particularly relevant for terminal tasks where episode length directly impacts latency and cost in deployment.

Convergence Rate: For GEPA, we track fitness improvement over optimization iterations to characterize sample efficiency. We report the number of metric evaluations required to reach 90% of final performance.

Generalization Gap: The difference between training set performance (tasks seen during optimization) and test set performance (held-out tasks). Large gaps indicate overfitting to the optimization set.

3.3.3 Baseline Specifications

RL Baselines (from GEM): We compare against three published baselines: REINFORCE+ReBN (REINFORCE with Return Batch Normalization, GEM’s recommended baseline for multi-turn tasks), PPO (Proximal Policy Optimization with learned value function), and GRPO (Group Relative Policy Optimization). All RL baselines are trained on Qwen3 models (1.7B, 4B parameters) with results taken directly from the GEM paper.

Prompt Optimization Baselines: To contextualize GEPA’s performance, we also compare against a zero-shot baseline (unoptimized DSPy agent with hand-written instructions) and MIPROv2 (DSPy’s prior-generation optimizer using Bayesian optimization).

Model Configurations: For primary experiments, we use Qwen3-1.7B and Qwen3-4B (matching GEM’s RL experiments) served via Amazon Bedrock. For generalization experiments, we evaluate on Claude Sonnet 4.5 via Bedrock.

3.4 Data Collection

Data collection occurs at three levels: task datasets from GEM, optimization trajectories during GEPA runs, and evaluation results on held-out test sets.

3.4.1 Data Sources

Task Datasets: We use the datasets bundled with GEM environments: Math12K (mathematical reasoning with verified solutions), CodeContest (competitive programming with test suites), and HotpotQA (multi-hop QA with ground-truth answers). Each dataset is pre-split into training, validation, and test partitions. GEPA optimization uses only the training partition; validation guides early stopping; test results are reported for final comparison.

RL Baseline Data: Performance metrics for RL baselines are extracted from the GEM paper’s published results and supplementary materials. Where specific configurations are ambiguous, we contact the authors for clarification or use the most conservative (lowest-performing) reported values.

3.4.2 Data Capture Methods

Trajectory Logging: Every episode interaction is logged with episode metadata (task ID, agent configuration, timestamp, random seed), per-turn data (observation, reasoning trace, action, reward, state delta), and episode summary (total return, success/failure, step count, duration).

GEPA Optimization Logs: For each optimization iteration, we capture the current prompt variant, fitness scores on the evaluation minibatch, reflection model output, Pareto frontier membership status, and token counts with API latency for cost accounting.

Evaluation Snapshots: At configurable intervals during optimization (every 10 iterations by default), we evaluate the current best prompt on the full validation set and log performance to track convergence curves.

Storage: All experimental data is stored in JSON format on Amazon S3, organized by run identifier, with code version-controlled in Git.

3.5 Sample and Sampling Procedures

3.5.1 Task Sampling

For each environment, we use the designated data partitions from GEM:

- **Training Set (GEPA Optimization):** Sample sizes vary by environment: Math12K uses a stratified subsample of 500 problems (from 12,000 total) due to API cost constraints; CodeContest uses the full training set (approximately 500 problems); HotpotQA uses a stratified sample of 500 problems.
- **Validation Set:** A held-out subset (10% of training data or designated validation split) is used for early stopping and hyperparameter selection. This set is never used to compute fitness during GEPA optimization.
- **Test Set:** The designated test partition from each GEM environment is used exclusively for final evaluation. Test set performance is computed once per experimental configuration to prevent data leakage.

3.5.2 Episode Sampling

- **Evaluation Rollouts:** Each test-set task is evaluated with $n = 5$ independent rollouts using different random seeds to account for stochasticity in LLM sampling. We report mean performance and standard error across rollouts.
- **GEPA Minibatch Sampling:** During optimization, GEPA evaluates candidate prompts on minibatches of tasks. Following GEPA defaults, we use minibatch size of 3 tasks per reflection step. Tasks are sampled uniformly from the training set without replacement within each optimization epoch.

3.5.3 Randomization

All experiments use fixed random seeds at three levels:

- **Data sampling seed:** Controls which tasks appear in training/validation/test splits (fixed across all experiments to ensure identical evaluation sets).
- **GEPA initialization seed:** Controls the random components of GEPA’s evolutionary process (varied across replication runs).
- **LLM sampling seed:** Where supported by the API, controls LLM decoding randomness (temperature, top-p sampling).

Each primary experimental configuration is run 3 times with different GEPA initialization seeds. Results are reported as mean $\pm$ standard error across replications.

3.6 Ethical Considerations

All task datasets are publicly available benchmarks with no personally identifiable information; IRB approval is not required. We commit to releasing the GEM-DSPy adapter library (MIT license), experimental configurations, and trajectory data.

3.7 Validity and Reliability

Internal Validity: To isolate the optimization method as the independent variable, we control for model architecture (identical Qwen3 models), evaluation environment (same GEM tasks), decoding parameters (matched temperature, top-p, token limits), and test sets. We mitigate confounds through sensitivity analysis on GEPA hyperparameters and by documenting API versions.

External Validity: Evaluating across three task types (math, code, QA) and a secondary model (Claude Sonnet 4.5) tests generalization. Results may not transfer to multi-modal tasks, embodied agents, or significantly different model scales.

Reliability: Fixed random seeds ensure reproducibility; each configuration runs 3 times with different seeds. Complete code, configurations, and raw results will be released. Baseline performance is verified against published benchmarks.

3.8 Methodological Limitations

- **Comparison Asymmetry:** We compare GEPA-optimized agents against published RL baselines rather than reproducing RL training ourselves. Any implementation errors on our side would disadvantage GEPA; we accept this conservative bias.
- **API Dependency:** LLM API variability (outages, rate limits, model updates) may introduce noise. We mitigate this by logging API versions and replicating critical experiments.
- **Cost Constraints:** API costs necessitate training set subsampling (500 tasks), which may reduce optimization quality compared to full datasets.

3.9 Additional Objectives (Stretch Goals)

If primary objectives are completed ahead of schedule, we will pursue: (1) **Ablation studies** varying GEPA components and comparing trajectory-aware vs. terminal-only fitness; (2) **Efficiency frontier analysis** to identify the cost crossover point between RL and GEPA; (3) **Transfer learning** tests across tasks and model families; and (4) **Expanded training samples** using fuller dataset sampling to assess optimization quality at scale.

3.10 Project Timeline

The project spans 16 weeks, organized into five phases with explicit milestones and deliverables. Project management and version control are maintained via GitHub, with feature branches for each major component and protected main branch requiring code review.

Phase	Weeks	Activities & Deliverables
Phase 1: Environment Setup	1–2	• Configure AWS infrastructure (Bedrock, S3) • Verify Qwen3 model access via Bedrock endpoints via custom import • Install and verify GEM environments locally • Set up DSPy with GEPA optimizer • Initialize GitHub repository with CI/CD pipeline • <i>Milestone:</i> Zero-shot baseline runs successfully on Math12K
Phase 2: GEM-DSPy-GEPA Integration	3–7	• Design and implement GEM environment wrapper for DSPy • Build trajectory capture and logging infrastructure • Implement trajectory-aware fitness functions with feedback generation • Integrate GEPA optimizer with GEM reward signals • Develop tool integration layer (Python execution, search) • Unit and integration testing across environment types • <i>Milestone:</i> GEPA optimization completes successfully on Math12K, CodeContest, and HotpotQA
Phase 3: Primary Experiments	8–11	• Run GEPA optimization on all target environments • Execute evaluation on held-out test sets • Collect cost, latency, and efficiency metrics • Run replications (3 seeds per configuration) • <i>Milestone:</i> Complete results for Qwen3 models across all environments

Phase	Weeks	Activities & Deliverables
Phase 4: Generaliza- tion & Ablations	12–13	• Run generalization experiments (Claude Sonnet 4.5) • Conduct hyperparameter sensitivity analysis • Perform ablation studies on fitness function components (if time permits) • <i>Milestone:</i> Complete results for secondary model configurations
Phase 5: Analysis & Writing	14–16	• Statistical analysis and hypothesis testing • Generate figures and tables • Draft results and discussion sections • Prepare code and data for open-source release • <i>Milestone:</i> Final paper submission and library release

Table 3.1: Project timeline with phases, activities, and milestones

3.10.1 Risk Mitigation

- **Integration Focus:** Phase 2 is allocated 5 weeks (one-third of the project) to ensure robust integration between GEM, DSPy, and GEPA. This extended timeline accounts for the novel nature of bridging RL gym infrastructure with prompt optimization frameworks.
- **Schedule Buffers:** Phase 3 includes flexibility for debugging integration issues that emerge during full-scale experiments. If primary experiments complete early, time is allocated to stretch goals.
- **Scope Reduction Triggers:** If Phase 2 extends beyond Week 7, we will reduce target environments from 3 to 2 (prioritize Math12K and CodeContest), reduce replications from 3 to 2 seeds per configuration, and defer generalization experiments to stretch goals.
- **Critical Path:** The minimum viable deliverable requires Phases 1–3 and 5. Phase 4 (generalization) is valuable but not essential for the core research question.

References

- Agrawal, L. A., Tan, S., Soylu, D., Ziemis, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M. J., Jiang, M., Potts, C., Sen, K., Dimakis, A. G., Stoica, I., Klein, D., Zaharia, M., & Khattab, O. (2025). GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. <https://arxiv.org/abs/2507.19457>
- AI, M. (2025). MLGym: A New Framework and Benchmark for Advancing AI Research Agents. <https://arxiv.org/abs/2502.14499>
- Bahdanau, D., Brakel, P., Xu, K., Goyal, A., Lowe, R., Pineau, J., Courville, A., & Bengio, Y. (2017). An actor-critic algorithm for sequence prediction. <https://arxiv.org/abs/1607.07086>
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., ... Kaplan, J. (2022). Constitutional AI: Harmlessness from AI Feedback. <https://arxiv.org/abs/2212.08073>
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. <https://arxiv.org/abs/1606.01540>
- Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2017). Deep Reinforcement Learning from Human Preferences. *Advances in Neural Information Processing Systems*, 30. <https://arxiv.org/abs/1706.03741>
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197. <https://doi.org/10.1109/4235.996017>
- DeepSeek-AI. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. <https://arxiv.org/abs/2501.12948>
- Fernando, C., Banarse, D., Michalewski, H., Osindero, S., & Rocktäschel, T. (2023). Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. <https://arxiv.org/abs/2309.16797>
- Ferret, J., Marinier, R., Geist, M., & Pietquin, O. (2023). A Survey of Temporal Credit Assignment in Deep Reinforcement Learning. <https://arxiv.org/abs/2312.01072>
- Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., & Yang, Y. (2024). Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers. <https://arxiv.org/abs/2309.08532>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*. <https://arxiv.org/abs/2310.06770>

- Kaelbling, L. P., Littman, M. L., & Cassandra, A. R. (1998). Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence*, 101(1-2), 99–134. [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X)
- Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., & Potts, C. (2024). DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *The Twelfth International Conference on Learning Representations*. <https://dsp.ai>
- Lakens, D. (2017). Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses. *Social Psychological and Personality Science*, 8(4), 355–362. <https://doi.org/10.1177/1948550617697177>
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023). Let’s Verify Step by Step. <https://arxiv.org/abs/2305.20050>
- Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., . . . Tang, J. (2024). AgentBench: Evaluating LLMs as Agents. *International Conference on Learning Representations*. <https://github.com/THUDM/AgentBench>
- Liu, Z., Sims, A., Duan, K., Chen, C., Yu, S., Zhou, X., Xu, H., Xiong, S., Liu, B., Tan, C., Beh, C. Y., Wang, W., Zhu, H., Shi, W., Yang, D., Shieh, M., Teh, Y. W., Lee, W. S., & Lin, M. (2025). GEM: A Gym for Agentic LLMs. <https://arxiv.org/abs/2510.01051>
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). Self-Refine: Iterative Refinement with Self-Feedback. <https://arxiv.org/abs/2303.17651>
- Marber, J., et al. (2024). LMRL-Gym: Benchmarks for Multi-Turn Reinforcement Learning with Language Models. *International Conference on Learning Representations*. <https://lmrl-gym.github.io/>
- Ng, A. Y., Harada, D., & Russell, S. (1999). Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping. *International Conference on Machine Learning*, 278–287. <https://people.eecs.berkeley.edu/~pabbeel/cs287-fa09/readings/NgHaradaRussell-shaping-ICML1999.pdf>
- NVIDIA. (2025). NeMo Gym: Build RL Environments for LLM Training [Open-source framework for reinforcement learning environments]. <https://github.com/NVIDIA-NeMo/Gym>
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems*, 35. <https://arxiv.org/abs/2203.02155>
- Pignatelli, E., Ferret, J., Geist, M., Mesnard, T., van Hasselt, H., Pietquin, O., & Toni, L. (2023). A Survey of Temporal Credit Assignment in Deep Reinforcement Learning. <https://arxiv.org/abs/2312.01072>
- Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerber, D., Duanmu, J., Barnes, S.,

- Xie, P., ... Li, J. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. <https://arxiv.org/abs/2307.16789>
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *Advances in Neural Information Processing Systems*, 36. <https://arxiv.org/abs/2305.18290>
- Ramesh, S. S., Hu, Y., Chaimalas, I., Mehta, V., Sessa, P. G., Ammar, H. B., & Bogunovic, I. (2024). Group robust preference optimization in reward-free rlhf. <https://arxiv.org/abs/2405.20304>
- Ranzato, M., Chopra, S., Auli, M., & Zaremba, W. (2016). Sequence level training with recurrent neural networks. <https://arxiv.org/abs/1511.06732>
- Salimans, T., Ho, J., Chen, X., Sidor, S., & Sutskever, I. (2017). Evolution strategies as a scalable alternative to reinforcement learning. <https://arxiv.org/abs/1703.03864>
- Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*, 36. <https://arxiv.org/abs/2302.04761>
- Schulman, J., Levine, S., Moritz, P., Jordan, M. I., & Abbeel, P. (2015). Trust Region Policy Optimization. *International Conference on Machine Learning*, 1889–1897. <https://arxiv.org/abs/1502.05477>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. <https://arxiv.org/abs/1707.06347>
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. <https://arxiv.org/abs/2402.03300>
- Sinii, V., Balagansky, N., Aksenov, Y., Kurochkin, V., Laptev, D., Gerasimov, G., Gorbatsovski, A., Shaposhnikov, B., & Gavrilov, D. (2025). Steering LLM Reasoning Through Bias-Only Adaptation. <https://arxiv.org/abs/2505.18706>
- Stiennon, N., Ouyang, L., Wu, J., Ziegler, D. M., Lowe, R., Voss, C., Radford, A., Amodei, D., & Christiano, P. F. (2020). Learning to Summarize with Human Feedback. *Advances in Neural Information Processing Systems*, 33, 3008–3021. <https://arxiv.org/abs/2009.01325>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd). MIT Press. <http://incompleteideas.net/book/the-book-2nd.html>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35, 24824–24837. <https://arxiv.org/abs/2201.11903>
- Xi, Z., et al. (2025). AgentGym-RL: Training LLM Agents for Long-Horizon Decision Making through Multi-Turn Reinforcement Learning. <https://arxiv.org/abs/2509.08755>
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. <https://arxiv.org/abs/2305.10601>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations*. <https://arxiv.org/abs/2210.03629>

- Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2023). WebArena: A Realistic Web Environment for Building Autonomous Agents. <https://arxiv.org/abs/2307.13854>
- Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., & Ba, J. (2022). Large Language Models Are Human-Level Prompt Engineers. <https://arxiv.org/abs/2211.01910>
- Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., Christiano, P., & Irving, G. (2019). Fine-Tuning Language Models from Human Preferences. <https://arxiv.org/abs/1909.08593>